

异动检测项目：源码导航与 ONNX Runtime 改造说明

1. 本文档的范围

本文说明工作区中的**项目自有代码**、当前算法调用链、可执行边界和 ONNX Runtime 改造路线。以下内容不属于业务代码，本文不逐行解释：

- `deps/`、`deps-cv4.4/`：第三方 OpenCV 头文件、库和许可证；
- `build/`、`build-ac1/`：历史 CMake 生成物、ELF 二进制和发布包；
- `.svn/`：版本控制元数据；
- 嵌套目录中的重复依赖和构建产物。

权威源码目录为 `imagecmp_v2/`。`imagecmp_v2/imagecmp_v2/` 在整理开始前是其业务源码的逐文件相同副本，应视为历史镜像，不应再同时维护两套代码。

2. 项目要解决的问题

系统以“实时巡检图”和“同一预置机位的正常样本图”为输入。上层业务系统先给出样本图中待检部件的 YOLO 框；算法库在实时图中定位该部件，再比较其结构，输出是否存在有效异动及异动区域。

它不是单图目标检测器，也不负责从全图自动识别部件类别。其最小输入为：

实时图路径
样本图路径
样本图中的 YOLO 框: [center_x, center_y, width, height]
深度结构相似度阈值

主接口的输出为：

返回码: 1=发现有效异动框, 0=未发现有效异动, 负数=调用/定位/运行时错误
simres: LDC + ResNet18 的全局结构相似度（并非每个返回分支都会正确赋值）
Box: 实时大图像素坐标中的异动框 [x, y, w, h]
liveRectYolo: 定位后的实时部件 YOLO 框
可选: 样本部件和实时部件 JPEG Base64

3. 当前实际算法链路

实时大图 + 样本大图 + 样本 YOLO 框

1. 从样本图裁出待检部件

2. 在实时图定位部件

OpenCV TM_CCOEFF_NORMED 模板匹配

不可信时: SuperPoint + LightGlue 特征匹配 + 单应性投影

- 3. LDC 提取两张部件图的轮廓图
- 4. ResNet18 计算轮廓图的全局特征余弦相似度
 - 高于调用方阈值：当前代码判无异动
 - 低于阈值：继续
- 5. ResNet18 稠密特征差异图
 - 平滑 → 阈值化 → 形态学处理 → 轮廓 → 合并/面积过滤
- 6. 输出最大有效异动框、裁剪图和可选 Base64

关键语义：定位与异动必须分开

模板匹配得分只能表达“样本部件在实时图的哪个位置、定位是否可信”。当前代码中，模板分数大于等于 0.85 会直接返回“无异动”，这是历史快速路径，可能漏掉小面积的螺栓缺失、指针偏移、颜色变化、局部破损等异动。

ONNX 改造时应调整为：

- 定位成功（模板匹配或特征匹配）
- 始终进入独立的结构/颜色/局部差异判定
 - 再输出“无异动”或“异动框”

4. 源码文件说明

文件	编译状态	主要职责
imagecmp_v2/CMakeLists.txt	当前主构建	构建 Ascend .om 版本动态库和测试程序；未构建 ONNX 文件。
imagecmp_v2/Makefile	历史备用	旧 Make 构建方式，路径假设和 CMake 不一致；已将 C++ 标准注释并修正为 C++14。
imagecmp_v2/imagecmp.h	公共头文件	Box、YOLO 转像素框、图像对比主接口等 C ABI 声明。
imagecmp_v2/yidong_main.cpp	当前主实现	模型池、接口实现、部件定位、LDC/ResNet 编排、结果输出。
imagecmp_v2/ascend_infer.h/.cpp	当前主实现	华为 ACL 初始化、设备 context/stream、.om 模型加载、内存和同步推理。
imagecmp_v2/image_matcher.h/.cpp	当前主实现	Ascend 版 SuperPoint + LightGlue：两图关键点匹配

文件	编译状态	主要职责
		和样本框到实时框的投影。
imagecmp_v2/ldc.h/.cpp	当前主实现	Ascend 版 LDC：多尺度边缘预测融合为轮廓图。
imagecmp_v2/deep_difference_detector.h/.cpp	当前主实现	Ascend 版 ResNet18：全局特征相似度与局部稠密差异框检测。
imagecmp_v2/imagecmp.cpp	当前主构建	传统 OpenCV 直方图、ORB 对齐、像素差、差异框合并、YOLO 转换、裁剪图填充。
imagecmp_v2/Laplacian.cpp	仅 CMake 当前构建	3x3 网格拉普拉斯清晰度检查，供上层重拍策略调用。
imagecmp_v2/base64.h/.c	当前主构建	JPEG 二进制与 Base64 的转换。
imagecmp_v2/test.cpp	当前主构建	固定路径、无限循环的人工压测程序，不是自动化回归测试。
imagecmp_v2/imagecmpfind.cpp	未被当前 CMake 构建	历史“图找图 + ORB + 像素差”方案，包含旧 <code>lxImageFind</code> 。
imagecmp_v2/deepseek_cpp_20250625_641801.cpp	未构建	早期、较简单的模糊检测备份。
imagecmp_v2/onnx/deep_difference_detector_onnx.cpp	<code>#if 0</code> 禁用	历史 ONNX Runtime 版 ResNet18 差异检测参考实现。
imagecmp_v2/onnx/image_matcher_onnx.cpp	<code>#if 0</code> 禁用	历史 ONNX Runtime 版 SuperPoint + LightGlue 参考实现。
imagecmp_v2/onnx/ldc_onnx.cpp	<code>#if 0</code> 禁用	历史 ONNX Runtime 版 LDC 参考实现。

5. 接口与构建现状

当前 Ascend 版本

`yidong_main.cpp` 硬编码加载以下模型：

```
superpoint_lightglue_pipeline_512x512_linux_aarch64.om
LDC_640x360.om
resnet18.om
```

它要求 Linux/AArch64、Ascend NPU、CANN/ACL、与构建匹配的 OpenCV 动态库。工作区中的 `liblxiimagecmp.so` 也是 AArch64 ELF，不能在当前 Windows 主机直接运行。

当前 ONNX 资产

工作区根目录已具备：

```
superpoint_lightglue_pipeline.onnx
LDC_640x360.onnx
resnet18.onnx
```

这三份文件目前**不会被当前主入口加载**：主实现依赖 `.om`，而 ONNX 参考文件被 `#if 0` 禁用，CMake 也没有查找或链接 ONNX Runtime。

已知接口问题

- `imagecmp.h` 声明了 `lxImageFind`，但当前 CMake 未编译其实现 `imagecmpfind.cpp`；不能视为稳定导出。
- `lxImage_MatcherDet`、`lxImage_DeepDifferenceDetector`、`lxLaplacian` 等实现存在，但未完整声明在公共头文件。
- `lxImageCmpOnnx` 在模板高分的快速返回分支没有写入 `simres`。
- `lxUninit()` 后若依赖自动初始化再次调用主入口，模型池可能无法通过 `once_flag` 自动重建。

6. ONNX Runtime 改造方案

目标是支持普通 x86 Windows/Linux 或 Linux AArch64 上的 ONNX Runtime 推理，不再强依赖 Ascend CANN。建议将后端设计成可选项：

```
IMAGECMP_BACKEND=ONNX      # 开发、x86 测试环境
IMAGECMP_BACKEND=ASCEND    # 现场 Ascend 环境（如仍保留）
```

阶段一：整理可编译的 ONNX 推理层

1. 新建或重构公共类接口，使 `ImageMatcher`、`LDC`、`DeepDifferenceDetector` 的 ONNX 实现与现有头文件一致。
2. 移除三个 ONNX 文件的 `#if 0`，但不要直接取消注释后混编：它们目前在 `.cpp` 内重复定义类，且与现有 Ascend 类冲突。
3. 引入 ONNX Runtime C++ API：头文件 `onnxruntime_cxx_api.h`、动态库/导入库、运行时 DLL/SO。
4. 在启动时读取并验证模型的输入输出名称、数据类型、形状。重点检查：
 - SuperPoint+LightGlue：`images` 输入及 `keypoints`、`matches`、`mscores` 输出；

- LDC：输入 [1,3,640,360] 或模型实际形状、所有边缘输出；
- ResNet18：输入 [1,3,224,224]、特征输出的空间尺寸与通道数。

阶段二：改造主编排

1. 用 ONNX Session 生命周期替换 AscendRuntime、AclModel 和 Ascend 模型池，或抽象为统一 InferenceBackend。
2. ModelPipeline 改为持有 ONNX 的三个推理器；保留“整套借出/归还”的并发控制。
3. 将模型路径改为配置项或初始化参数，不再硬编码 .om 文件名。
4. 修改 lxiImageCmpOnnx：模板匹配只用于定位；定位成功后必须执行独立异动判定，不能以 0.85 模板分数直接返回无异动。
5. 补齐公共头文件，定义清晰的返回码、输出字段、内存释放函数和线程安全约定。

阶段三：构建与发布

1. 在 CMake 中添加 find_package(onnxruntime CONFIG REQUIRED) 或显式的 ONNXRUNTIME_ROOT 配置。
2. 支持 Windows DLL 和 Linux SO 的运行时搜索路径配置。
3. 将模型、ONNX Runtime 运行库、OpenCV 运行库与测试说明组成可复制的发布包。
4. 保留 Ascend 后端时，确保两种后端的输入输出和判定语义一致。

7. 测试计划

冒烟测试

新增的 test/ 目录包含 live.jpg、temp.jpg 和 yolo.txt。测试程序应改为命令行形式：

```
imagecmp_test --live test/live.jpg --template test/temp.jpg --yolo test/yolo.txt
```

每个 YOLO 框至少输出：定位方式、定位框、模板定位得分、结构相似度、返回码、异动框、耗时。

必须覆盖的回归样本

类别	期望结果
正常同机位图	无异动、无异动框。
轻微光照变化	不应误报。
图像模糊	清晰度模块识别并提示重拍。
机位偏移/旋转	模板匹配失败时由特征匹配定位成功。
小面积真实异动	发现异动并输出接近真实位置的框。
大面积遮挡/错位	返回定位异常或低可信结果，不能伪造局部异动。
呼吸器颜色变化	如要纳入需求，需单独定义颜色判定与标注标准。

验收指标

在有人工标注的验证集上统计：定位成功率、异动召回率、精确率、误报率、漏报率、框 IoU、单部件耗时和并发吞吐。未完成这一步前，不能用“模型能运行”替代“算法可上线”。

8. 下一步执行顺序

1. 获得或安装与当前目标系统匹配的 ONNX Runtime 开发/运行包和 C++ 构建工具。
2. 先为三份 ONNX 文件编写独立模型加载与单次推理校验，确认节点名、形状和输出语义。
3. 完成 ONNX 后端重构和命令行测试程序。
4. 使用 `test/` 样例完成冒烟测试，再扩展为带真值标签的回归集。
5. 最后根据验证数据校准定位、相似度和差异面积阈值。